

CEE6501 Final Course Project

The final course project is intended to mirror and extend the programming assignments completed throughout the semester.

In the same spirit as the prior assignments, you will implement a structural analysis program using the direct stiffness

method (DSM). For the final project, your program must be able to analyze both 3D truss elements and 3D frame elements

within the same codebase, so that the user can specify the type of each element in the structural model.

Your final submission will take the form of a GitHub repository, which will serve as the complete package for your

project. You will submit a link to this repository.

There will be three main components in the submitted repository:

1. Python code
2. Final report
3. Final presentation

The final presentation will be given to the class on the final day of classes: April 24.

Deliverables and Dates

The final course project is worth 45% of your final grade.

The breakdown is as follows:

Code — 10%

This portion will evaluate:

- quality of GitHub repository organization and presentation
- clarity and modularity of the code structure
- whether the code runs correctly

- whether another user can follow your documentation and use the program successfully

Report — 20%

- Preliminary plan and progress submission — 5% — Due 04/10
- Final report — 15% — Due 04/24

Final Presentation — 15%

- Instructor evaluation — 10%
- Student evaluation — 5%

Preliminary Plan Deliverable

The preliminary deliverable is due on 04/10, approximately two weeks after the project is posted.

The purpose of this submission is to demonstrate that you have made a reasonable start on the project and have a clear plan for completing it successfully.

At minimum, this submission should include:

- a GitHub repository that has already been created
- a brief description of how you intend to organize your code
- a short discussion of the different components of your project
- a preliminary description of the structure you intend to model in the final part of the project
- a brief literature review or background summary showing that you have identified a structure for which sufficient information is available

This preliminary submission should take the form of a short write-up of approximately 3 pages.

The goal is to show that:

- you have begun organizing the project in a professional way
- you have a reasonable plan for how the code will be structured
- you have identified an interesting structure to study
- you have found enough information on that structure, whether from academic papers, technical reports, drawings, or

other credible sources, to support a meaningful analysis

In other words, by the preliminary submission stage, I want to see that your final modeling study is feasible and that you are positioned to complete the project successfully.

1. Python Code and GitHub Repository

Your code must be written in Python and organized in a way that is clear, modular, and easy for another user to download and run. The repository should reflect good coding practice, with the main script at the top level and helper functions organized into separate files or folders as appropriate.

At a high level, your code should contain three major parts:

- A. User input / problem specification
- B. Analysis loop / DSM implementation
- C. Output / results presentation

A. User Input

Your program should take as input a text file or `.json` file that the user provides when running the code. This input

file should contain all information needed to define the structural model, including, as applicable:

- node coordinates
- element connectivity
- element type designation (`3D_truss` or `3D_frame` , for example)
- material properties
- section properties
- support conditions
- nodal loads
- member loads
- prescribed support displacements
- any release information for frame elements

The goal is that a user should be able to define a full structural problem by editing this input file, without needing to modify the source code itself.

Your program should then preprocess this file and convert the user-defined information into data structures that can be used in the analysis loop.

How you set up the input format is up to you. For example, you may choose to define nodes, elements, loads, and supports directly in a .json file. You may also choose to create helper scripts that generate these input files automatically.

For instance, you might define a structural grid and spacing and then automatically generate node coordinates and connectivity from those inputs.

The key requirement is that your input format should be logical, documented clearly, and usable by someone other than yourself.

A very simple example of what this might look like is shown below.

```
{
  "nodes": {
    "1": [0.0, 0.0, 0.0],
    "2": [4.0, 0.0, 0.0]
  },
  "elements": {
    "1": {
      "type": "3D_truss",
      "nodes": [1, 2],
      "E": 200000000000.0,
      "A": 0.003
    }
  },
  "nodal_loads": {
    "2": [1000.0, 0.0, 0.0, 0.0, 0.0, 0.0]
  },
  "releases" : {}
  "etc...."
}
```

Your preprocessing step might then read this file and reorganize it into dictionaries, arrays, or class objects that are easier to use during assembly and solution.

```
import json

def load_model(filename):
    with open(filename, "r") as f:
        model = json.load(f)

    nodes = {int(k): v for k, v in model["nodes"].items()}
    elements = {int(k): v for k, v in model["elements"].items()}
    #ETC...

    return nodes, elements
```

B. Analysis Loop

Your analysis code should implement the direct stiffness method in a general way so that the same program can handle structural models made of mixed element types.

At minimum, your code should be able to:

- analyze structures containing both 3D truss and 3D frame elements
- allow the user to specify the element type for each member
- implement member end releases for frame elements using any valid method of your choosing
- implement member loading for frame elements
- implement prescribed support displacements
- implement temperature loading (uniform only)
- implement fabrication error (length error only)

To avoid the additional complexity associated with the most general 3D frame rotation formulation, we will make the following simplifying assumption:

All frame elements may be treated as bar elements with equal second moment of area in all bending directions.

With this assumption, you do not need to implement the more general 3D frame transformation involving a roll angle.

Instead, you may use the same geometric rotation matrix approach discussed in class for 3D truss elements, extended appropriately for the frame formulation used in class.

C. Output and Results Presentation

Your code should produce output in a form that is clear, organized, and genuinely useful to the user. A correct solver is important, but so is the ability to understand and interpret the results it produces.

At minimum, your output should clearly communicate key results such as:

- nodal displacements
- support reactions
- element force results
- any other important response quantities relevant to your implementation

Rather than only printing results to the screen, your program should also save output files in a logical location within your repository. For example, you might create an `outputs/` folder for each example or analysis case, and write the results there in a format that is easy to review later.

You are encouraged to think carefully about how results should be presented and organized. The goal is not only to produce correct numerical output, but also to communicate the structural response in a way that is clear and easy to interpret.

For example, your code might:

- write a clean summary of key results to a `.txt` file
- save detailed numerical output to a `.json` file
- generate formatted tables that are easy to review
- output plots, figures, or screenshots that help visualize the structural behavior
- save separate result files for different example problems or load cases
- include simple visualizations of the structure, such as color-coding nodes by displacement magnitude with an accompanying legend

- highlight members experiencing larger axial forces, moments, or other important internal force resultants

Think carefully about what types of output would be most useful to someone trying to understand the behavior of the structure. Clear formatting, sensible file organization, and informative visualizations will strengthen the overall quality of the project.

The goal is not just to produce numbers, but to present those numbers in a way that another user can quickly understand.

Think about how you would want to receive results from a structural analysis program you were using yourself.

Be thoughtful and creative here. Well-organized outputs, clearly named files, and useful visualizations will make your project significantly stronger.

Code Organization

Your program should run from a main Jupyter notebook file (`.ipynb`) located in the top level of the repository. This notebook should serve as the primary entry point for the project and should call functions stored elsewhere in the repository.

You may organize and import your helper functions in whatever way you prefer, but your repository must be fully self-contained. In other words, the notebook should only call files, modules, and functions that are included within the repository itself. Do not rely on local file paths outside the repository, personal directory structures on your computer, or external code that is not included in the submitted repo.

A user should be able to clone or download your repository, follow your instructions, and run the project without needing to modify hard-coded paths or reconnect missing code dependencies.

You are encouraged to organize your code into helper modules such as:

- preprocessing / input parsing

- element stiffness and force routines
- assembly and partitioning
- boundary condition handling
- solving
- postprocessing / output formatting

The exact organization is up to you, but the repository should be structured in a logical and readable way.

GitHub Expectations

Because the final submission is a GitHub repository, it should be set up professionally. At minimum, your repository should include:

- all Python source code
- the input files used for your validation studies and final example
- a `README.md` file explaining the purpose of the repository and how to run the code
- the final report
- the final presentation

Your repository should be easy for another user to download, understand, and execute.

A possible repository structure is shown below:


```
final-project-repo/
|
├─ main.ipynb
├─ README.md
├─ requirements.txt
|
├─ inputs/
|   │ └─ validation_truss.json
|   │ └─ validation_frame.json
|   └─ final_structure.json
|
├─ outputs/
|   └─ ...
|
├─ helpers/
|   │ └─ preprocess.py
|   │ └─ elements.py
|   │ └─ assembly.py
|   │ └─ solver.py
|   └─ postprocess.py
|
├─ report/
|   └─ final_report.pdf
|
├─ presentation/
|   └─ final_presentation.pptx
|
└─ examples/
    │ └─ validation_results/
    │ └─ final_structure_results/
    ...
```

README File

The `README.md` file should provide enough information for a user, in this case me, to run your code successfully.

At minimum, the README should explain:

- what the code does
- how to install any dependencies
- how to run the main script

- how to prepare an input file
- what outputs to expect
- how to reproduce your example problems

In addition to verifying that your included test files run correctly, I may also create my own structural input file

using your documented input format and test your program on that structure. I will follow the instructions in your

README, and I expect the code to run correctly and produce reasonable output.

For that reason, the clarity and completeness of your documentation matter.

2. Final Report

The final report should combine the following three components:

- A. User guide for your code
- B. Technical validation of your implementation
- C. Modeling study of a complex 3D structure

A. User Guide

The first part of the report should explain clearly how to run your program. This should include:

- the required Python version and any packages used
- how the repository is organized
- how to run the main script
- how the input file is structured
- what outputs the code produces
- how a user can modify the input file to analyze a different structure

This section should be written so that another student could reasonably use your code without needing additional explanation.

This portion of the report will overlap with the information in your `README.md`, but it should go into greater detail.

The README should provide high-level instructions, whereas the report can give a fuller explanation

of how the code is structured and used.

B. Validation Examples

The next part of the report should demonstrate that your program is working correctly.

For validation, you should use your code to analyze simple structures for which you already know the correct answer.

Specifically, you should include:

- one simple 3D truss example
- one simple 3D frame example

These may be structures you have already analyzed earlier in the course, provided that you document the hand solution

carefully and are confident that it is correct. You may also use problems from the textbook, but you are still required

to show your own hand-worked solution as part of the report.

For the 3D truss validation case, the example should be simple, but it should still be rich enough to demonstrate the

key features of your truss implementation. In particular, your validation example should, where applicable, demonstrate

that your code can correctly handle:

- support displacements
- temperature loading
- fabrication error loading

For the 3D frame validation case, the example should be rich enough to demonstrate the main features your code is

intended to handle. In particular, it should, where applicable, demonstrate:

- frame behavior
- member releases
- support displacements
- member loading
- temperature loading
- fabrication error loading

The goal of these validation examples is not simply to show that your code runs, but to show that it correctly captures the different types of behavior and loading conditions your implementation is supposed to support.

Both the truss and frame validation examples does not need to be hand-solved as a fully three-dimensional problem. A two-dimensional problem solved by hand is acceptable, as long as you model it within your 3D code framework.

Conceptually, a 2D frame is just a special case of a 3D frame in which all geometry and loading lie in a single plane, and the out-of-plane degrees of freedom are restrained so that the structure cannot deform out of that plane. In other words, your code may still treat the model as a 3D structure, but the appropriate boundary conditions should be applied so that its response is restricted to the intended 2D behavior.

If you choose this approach, make sure you clearly explain how the 2D problem is embedded within the 3D model and which restraints are used to suppress the out-of-plane motion and rotations.

For each validation example, your report should include:

- a description of the structure
- the input data
- the hand solution
- the corresponding computer-model input
- the code results
- a comparison between the hand solution and the code output

The goal of this section is to show clearly and convincingly that your implementation is correct.

C. Final Complex Structure Study

The final part of the report should be a more substantial modeling study of a complex structure of your choosing.

This component is intended to mirror the spirit of Assignment A5, where you used your code to analyze a more complex truss structure. For the final project, you will instead analyze a complex 3D structure that uses a mix of

truss and
frame elements.

You may choose a real structure that is primarily a frame and then idealize some members as truss elements if needed.

The main goal is to demonstrate that your program can successfully analyze a structure containing mixed element types.

Background Research and Literature Review

Your report should begin the final structure study with a careful write-up on the chosen structure.

You should try to gather as much information as possible, including:

- overall structural configuration
- geometry
- likely load paths
- material assumptions
- boundary/support assumptions
- photographs, drawings, or diagrams if available
- published reports, technical documents, or academic papers related to the structure
- any previous analyses of the structure, if available

The more information you can find, the better. Because of this, you should think carefully about the structure you choose.

Modeling and Results Discussion

After introducing the structure, your report should explain clearly how you translated the real structure into an analytical model in your code.

This discussion should include:

- a clear description of the modeling assumptions
- the overall idealization of the structure
- where truss elements were used and where frame elements were used
- support and boundary condition assumptions
- load assumptions
- section and material properties

- the actual input file used in your program

You should then present the results of your analysis and discuss them critically. The goal is not simply to show that the code produced output, but to demonstrate that you can interpret that output in a thoughtful and engineering-informed way.

Your discussion might address questions such as:

- where do the largest displacements occur, and does that make physical sense?
- which members carry the largest axial forces, shear forces, torsional moments, or bending moments?
- where are the most highly stressed or most heavily loaded regions of the structure?
- do the results appear physically reasonable given the structural form and expected load path?
- how sensitive are the results to your modeling assumptions?
- how do the results change if you alter support conditions, member idealizations, or loading assumptions?
- do certain alternative assumptions appear more realistic than your original model?

You are also encouraged to explore a small number of meaningful “what-if” studies that help reveal how the structure behaves under different scenarios. For example, you might investigate questions such as:

- What happens if there is an extreme temperature drop, such as a historic freeze of -30°C ?
- What happens if a subset of members is fabricated incorrectly?
- How large of a temperature change would be required for a particular member force to exceed some critical threshold?
- What happens if elevated temperature is introduced in a localized region, such as a simplified fire scenario?
- What if adjacement construction causes a subset of supports to settle by a certain amount?
- Which assumptions most strongly influence the structural response?

Do not simply present output tables. The strongest reports will go beyond reporting numbers and instead explain what the results suggest about the behavior of the structure, the limitations of the model, and the consequences of different assumptions.

The more thoughtful, critical, and exploratory your discussion is, the stronger this part of the project will be.

3. Final Presentation

Your final presentation should be a PowerPoint presentation delivered to the class.

The total time allotted will be 15 minutes, which should be planned approximately as:

- 12 minutes of presentation
- 3 minutes for questions and setup/transition time

Presentation Content

Your presentation should provide a concise but complete summary of your final project.

It should begin by documenting your code and explaining how it is organized. In particular, you should describe:

- how your program is structured
- how the user inputs data
- how you organized the code
- key design decisions you made
- any important implementation choices in your DSM workflow

You should then introduce the complex structure that you selected for your final study.

In this section, summarize:

- the structure itself
- why you chose it
- what information you found in your literature review
- what modeling assumptions you made

Finally, present the results of your analysis of that structure.

This portion of the presentation should summarize the key findings from your report, including:

- the structural behavior you observed
- the most important response quantities

- what your results suggest about the structure
- how modeling assumptions influenced the results
- discuss some of your “what-if” scenarios

In other words, the presentation should be a concise presentation of the main findings of your final report.

Presentation Style

Because the presentation time is limited, you should be selective and focus on the most important aspects of the project. Do not try to reproduce every detail from your report. Instead, aim to communicate the main ideas clearly and efficiently:

- what your code does
- how you structured it and the key design decisions you made
- how you validated the implementation
- what structure you chose to study
- what you learned from analyzing it

A strong presentation will be clear, well-organized, and intentional in what it emphasizes. Prioritize the material that best communicates the quality of your work and the main findings of your project.

Use figures and visual aids as much as possible. Well-chosen diagrams, structural models, screenshots, and result visualizations will make the presentation much stronger, especially when they are generated directly from your code. Ideally, your code should help produce not only numerical results, but also visuals that you can use to communicate those results effectively.

More broadly, this is also good practice for future technical work. Well-designed code can support the creation of clean, reusable figures for presentations, conference papers, and journal articles. Thoughtful output and visualization are therefore not just helpful for this class presentation, but are part of good computational and research workflow more generally.